

TECHNICAL DOCUMENTATION

Backend API Layer

Technical Specification

FastAPI HTTP surface, authentication, persistence, Alembic ownership, and AI orchestration — covering auth, profile and program/session CRUD, AI-backed routes, live coaching, and history-driven personalization.



PROJECT

AI Fitness Coach

MODULE

backend/app/ (routers, schemas, models)

DOCUMENT VERSION

1.0

LAST UPDATED

July 2026

TABLE OF CONTENTS

Contents

Contents	2
1.1 What Was Delivered	4
2.1 Where the API Sits in the Product	5
2.2 Client and Stack	5
3.1 Ownership Matrix	5
3.2 Boundary Rules	6
5.1 Design Principles	7
6.1 Milestone 1 — Auth and Identity	8
6.2 Milestone 2 — Profile and Domain CRUD	8
6.3 Milestone 3 — AI Program Generation & Suggest-Next	8
6.4 Milestone 4 — Session Save + History Indexing	9
6.5 Milestone 5 — Live Coach HTTP Surface	9
6.6 Milestone 6 — Profile Field & Contracts	10
8.1 users / user_profiles	11
8.2 programs / program_exercises	11
8.3 sessions / session_exercises	11
8.4 AI-Related Tables	11
10.1 Endpoint Catalog	12
10.2 Recommended Client Session Flow	13
10.3 Common Status Codes	14
12.1 Representative Migration Chain	15

14.1	Environment	16
14.2	Local Stack	16
15.1	Completed (Person B)	16
15.2	Remaining (Product)	17

CHAPTER 01

1 Executive Summary

AI Fitness Coach is designed as a complete digital coaching loop: it learns about the user, proposes a training plan, guides the athlete through the workout in real time, records what actually happened, and adapts the following session accordingly. The AI/RAG team (Person A) delivers this intelligence as a set of Python services. This specification documents how the API/data team (Person B) exposes those services as a secure, owner-scoped HTTP interface that the Flutter mobile client (Person C) can consume.

The scope of this document is the backend API and data layer: JWT-based authentication, profile and program/session CRUD, AI-backed generation and live-coaching routes, Alembic migration ownership, and the mechanism by which completed sessions feed the personal-history memory used for future recommendations.

8CAPABILITIES
DELIVERED**18**

HTTP ENDPOINTS

6

DELIVERY MILESTONES

7

ALEMBIC REVISIONS

1.1 What Was Delivered

TABLE 1.1 — DELIVERED CAPABILITIES

Capability	Purpose
Auth & identity	Registration and login, bcrypt password hashing, JWT Bearer access tokens.
Profile API	Read and partial-update of the fitness profile, scoped to the authenticated JWT user.
Programs & sessions	Persistence of training plans and completed workouts with strict ownership checks.
AI program routes	generate and suggest-next endpoints validate AI output and persist it under the JWT user.
Live coach routes	Start / after-exercise / mid-session / end turns exposed as text-in / text-out HTTP calls.
History wiring	After POST /api/sessions succeeds, a best-effort call indexes the session into personal history.
Migrations	Ownership of the Alembic chain: users, programs, AI-related tables, and the profile time field.
Contracts & tests	Mobile-facing api-contracts.md plus a pytest suite covering every Person B route.

■ NOTE

Companion references: **docs/api-contracts.md** is the primary reference for Person C (request/response detail), and **docs/AI_COACHING_LAYER.md** documents the AI internals owned by Person A.

CHAPTER 02

2 Product Vision — Role of the API

2.1 Where the API Sits in the Product

The HTTP API is the connective tissue between the mobile client and the AI services. Its responsibilities differ across the four phases of the coaching lifecycle:

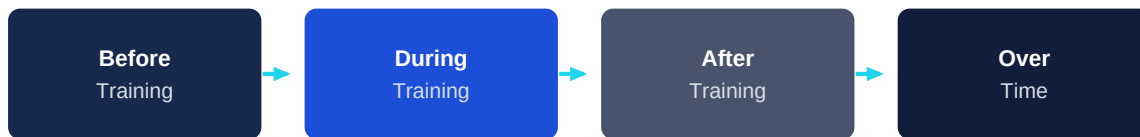


Figure 2.1 — Coaching lifecycle and the HTTP surface active in each phase

TABLE 2.1 — HTTP RESPONSIBILITY BY PHASE

Phase	HTTP Responsibility
Before training	Auth, profile PATCH, POST <code>/api/programs/generate</code> or <code>suggest-next</code> , list/get programs.
During training	Live coach turns (<code>/api/coach/*</code>) returning structured messages and UI prompts; no auto-save mid-workout.
After training	POST <code>/api/coach/end</code> → client maps the snapshot → POST <code>/api/sessions</code> , which indexes history.
Over time	POST <code>/api/programs/suggest-next</code> reads from the indexed personal history to adapt future plans.

2.2 Client and Stack

Mobile Client Flutter (planned), Person C. Consumes this HTTP API exclusively.	API Layer FastAPI, Python 3.12, Pydantic schemas, SQLAlchemy async engine. Person B.	AI Services backend/app/ai/, Person A. Called from routers, never exposed as routes.	Database PostgreSQL 16 with the pgvector extension, orchestrated via Docker Compose.
--	--	--	--

CHAPTER 03

3 Team Responsibilities

3.1 Ownership Matrix

TABLE 3.1 — OWNERSHIP BY ROLE

Role	Ownership
Person A — AI / RAG	backend/app/ai/, AI-related model schemas, ingestion, retrieval, generation, live coach logic, smoke scripts.
Person B — API / data	Routers, auth, HTTP schemas, persistence, Alembic apply/review, wiring AI calls from routes, API docs and route tests.
Person C — Mobile	Flutter UI, optional voice STT/TTS layered on top of the same text-based APIs.

3.2 Boundary Rules

★ IMPORTANT

No FastAPI routes live inside **app/ai/**. Routers live exclusively under **app/routers/** and call into AI functions — the AI package is never route-addressable.

★ IMPORTANT

The JWT user is the single source of truth for ownership. The API never trusts a client-supplied **user_id** in a request body for persistence decisions.

■ NOTE

Coach routes are AI-only and never persist. Persistence of a finished workout happens exclusively through **POST /api/sessions**.

■ NOTE

Voice input/output stays entirely on the client. Every API payload is text (plus structured scales); STT/TTS wrapping is Person C's responsibility.

CHAPTER 04

4

Foundation

Before AI orchestration work began, the backend already provided a functioning core on which the AI-backed and live-coaching features were layered:

TABLE 4.1 — PRE-EXISTING FOUNDATION

Foundational Capability	Description
User registration / login	JWT-based identity, already wired before AI integration.
Fitness profiles	Structured profile fields used later for AI personalization.

Foundational Capability	Description
Programs with ordered exercises	Planned workouts with sets, reps, rest, and ordering.
Sessions linked to programs	Completed workouts referencing an optional program_id.
Session feedback fields	feeling, fatigue, difficulty, skipped, and free-text notes.

■ IMPLEMENTATION DETAIL

Person B's delivery work orchestrates Person A's AI services on top of this existing foundation, producing a complete, mobile-ready contract rather than rebuilding domain persistence from scratch.

CHAPTER 05

5 Architecture Overview

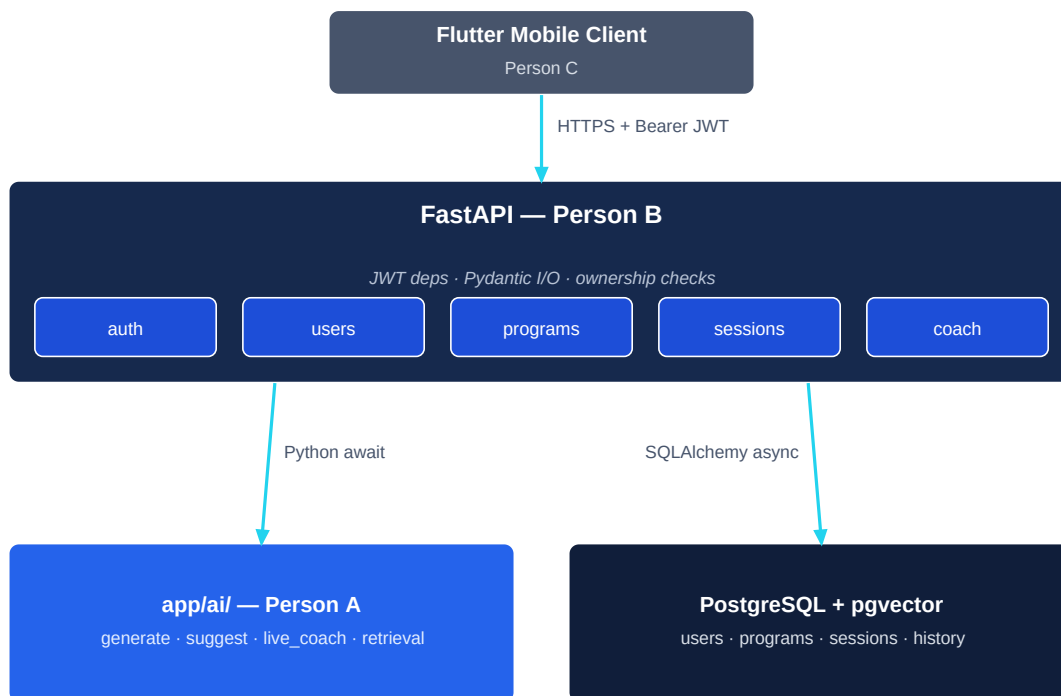


Figure 5.1 — System architecture: client, API, AI services, and persistence

5.1 Design Principles

Owner-scoped reads/writes. List endpoints filter by the JWT user; a get-by-id on a row that exists but isn't owned returns 403, not 404, preserving existence semantics without leaking cross-user data.

Profile completeness for AI. generate, suggest-next, and coach routes require the profile's required fields to be present; an incomplete profile yields 400 Incomplete profile before any AI call is made.

Validate AI before persist. Program AI output is parsed and validated as ProgramCreate prior to commit. Invalid AI output returns 502 and guarantees no partial write.

Best-effort history indexing. A session save commits first; indexing into history runs afterward and its failure is logged without rolling back the already-saved session.

Stable HTTP contracts. Detailed request/response shapes live in api-contracts.md, and the generated OpenAPI schema at /docs is kept in sync with the router implementations.

CHAPTER 06

6 Delivery Milestones

The API layer was delivered across six milestones, each adding a coherent capability on top of the previous one.

6.1 Milestone 1 — Auth and Identity

What: Registration, login, and the JWT dependency used across protected routes.

Why: Every coaching and CRUD route must be attributable to exactly one user.

TABLE 6.1 — MILESTONE 1 COMPONENTS

Component	Responsibility
<code>routers/auth.py</code>	POST /api/register, POST /api/login
<code>core/security.py</code>	Password hashing
<code>core/deps.py</code>	get_current_user via OAuth2PasswordBearer (tokenUrl=/api/login)

6.2 Milestone 2 — Profile and Domain CRUD

What: Profile get/patch; program and session list/get/create with ownership enforcement.

Why: Mobile needs to persist plans and completed work both before and after AI involvement.

TABLE 6.2 — MILESTONE 2 COMPONENTS

Component	Responsibility
<code>routers/users.py</code>	GET / PATCH /api/me/profile
<code>routers/programs.py</code>	Create, list mine, get by id, list sessions for a program
<code>Alembic</code>	programs/sessions tables, program_id column on sessions

6.3 Milestone 3 — AI Program Generation & Suggest-Next

What: HTTP wrappers that load the DB profile, call the AI layer, and persist the result under the JWT user.

Why: Person C never talks to the underlying LLM provider directly — Person B owns orchestration and storage.

TABLE 6.3 — AI PROGRAM ROUTES

Route	AI Call	Persist
POST /api/programs/generate	generate_program	Yes → ProgramResponse 201
POST /api/programs/suggest-next	suggest_next_program	Yes → program + rationale + adaptations

◆ DESIGN DECISION

Profile fields used for AI context are always read from the database, never accepted as a client-supplied profile blob. Only an optional **start_date** may be sent in the request body.

6.4 Milestone 4 — Session Save + History Indexing

What: POST /api/sessions validates the optional program_id and exercise names, saves the session, and then indexes it into history.

Why: Suggest-next and coaching continuity depend on user_history_chunks being populated.

✓ BEST PRACTICE

After a successful commit, **index_session_history(...)** runs best-effort. Indexing errors are logged and never undo the already-saved session — a deliberate design decision to prioritize data durability over feature completeness.

6.5 Milestone 5 — Live Coach HTTP Surface

What: Four JWT-protected coach turns mirroring Person A's live_coach functions.

Why: In-session coaching is a core product differentiator; Flutter needs stable endpoints and prompt schemas.

TABLE 6.4 — LIVE COACH ROUTES

Route	AI Function	Persists?
POST /api/coach/start	start_session_check_in	No
POST /api/coach/after-exercise	after_exercise_feedback	No
POST /api/coach/mid-session	mid_session_coach	No
POST /api/coach/end	end_session_coach	No — returns a snapshot for the client to POST to /api/sessions

■ NOTE

An optional **program_id** on start / after-exercise loads an owned program to supply upcoming or planned exercise context to the coach.

6.6 Milestone 6 — Profile Field & Contracts

What: available_time_minutes on profiles; full mobile contracts; unit tests.

Why: Person C needs documented, test-backed APIs before building UI against them.

TABLE 6.5 — MILESTONE 6 ARTIFACTS

Artifact	Detail
Alembic revision	e7f8a9b0c1d2_add_available_time_minutes_to_user_profiles.py
Docs	docs/api-contracts.md
Tests	backend/tests/test_*.py — profile, generate, suggest-next, sessions, coach

CHAPTER 07

7 Package Layout

The backend package is organized so that each directory has a single, well-defined owner and responsibility:

```

backend/
  app/
    main.py      # FastAPI app; mounts routers; GET /health
    db.py        # Async engine / session
    core/
      config.py  # Env (SECRET_KEY, DATABASE_URL, Groq, ...)
      security.py # Password hashing / JWT helpers
      deps.py    # get_current_user
      models/    # SQLAlchemy: user, program, session, AI tables
      schemas/   # HTTP Pydantic I/O (auth, user, program, session, coach)
      routers/
        auth.py
        users.py
        programs.py # CRUD + generate + suggest-next
        sessions.py # CRUD + history index after create
        coach.py    # Live coach turns
      ai/          # Person A (called, not owned by B routes)
    alembic/versions/ # Migration chain (B owns apply/review)
    tests/         # pytest for B HTTP behaviour
    entrypoint.sh  # Wait for DB → alembic upgrade → uvicorn
    requirements.txt

```

Each folder exists to isolate a single concern: **core/** owns configuration and security primitives so they are never duplicated across routers; **models/** and **schemas/** are kept separate so the database representation can evolve independently of the HTTP contract; and **routers/** is the only layer permitted to declare FastAPI paths, which is what keeps **app/ai/** strictly a callable library rather than a second API surface.

CHAPTER 08

8 Data Model — API-Owned Domain

8.1 users / user_profiles

TABLE 8.1 — IDENTITY ENTITIES

Entity	Role
users	id, unique email, hashed_password
user_profiles	1:1 fitness fields used for coaching personalization

Profile fields consumed by AI (as **ProfileContext**): name, age, sex, height_cm, weight_kg, fitness_level, primary_goal, training_frequency, available_equipment, limitations.

■ WARNING

available_time_minutes is stored and exposed on profile GET/PATCH, but it is not currently passed into ProfileContext. This is a known gap relative to future product polish, not an oversight in the current milestone scope.

8.2 programs / program_exercises

Planned workouts: name, goal, dates, status, and an ordered list of exercises (sets, reps, rest, notes, order). Every program row is always tied to a user_id.

8.3 sessions / session_exercises

Completed workouts: start/end times, an optional program_id, feeling/fatigue/comments, and per-exercise completion rows. When program_id is set, each exercise_name on the session must exist on that program — enforced at the API layer before commit.

8.4 AI-Related Tables

Schema authored by Person A; migrations owned and applied by Person B:

TABLE 8.2 — AI-RELATED TABLES

Table	Role
knowledge_chunks	Global RAG vectors
exercises	Canonical exercise catalog
user_history_chunks	Per-user session memory for suggest-next

CHAPTER 09

9

Authentication and Security

TABLE 9.1 — SECURITY BEHAVIOUR

Topic	Behaviour
Token	JWT sent as Authorization: Bearer <access_token>
Obtain	POST /api/login with email/password returns access_token
Dependency	get_current_user enforced on every protected route
Ownership	Persistence always uses current_user.id; a cross-user resource lookup returns 403
Secrets	SECRET_KEY and DB credentials are supplied via .env and never committed
AI keys	GROQ_API_KEY is required at runtime for AI-backed routes

★ **IMPORTANT**

The only public (no-JWT) routes in the entire API are **POST /api/register**, **POST /api/login**, and **GET /health**. Every other route requires a valid Bearer token.

CHAPTER 10

10 HTTP API Surface

Full request/response examples, status codes, and curl samples live in docs/api-contracts.md. The interactive OpenAPI schema is always available at **/docs** on a running instance.

10.1 Endpoint Catalog

TABLE 10.1 — ENDPOINT CATALOG

Method	Path	Role
POST	/api/register	Create user + profile
POST	/api/login	Issue JWT
GET	/api/me/profile	Read profile
PATCH	/api/me/profile	Partial update
POST	/api/programs	Manual program create
POST	/api/programs/generate	AI generate + persist
POST	/api/programs/suggest-next	AI next + persist
GET	/api/me/programs	List own programs
GET	/api/programs/{id}	Get owned program
GET	/api/me/programs/{id}/sessions	Sessions for one program
POST	/api/sessions	Save session (+ index history)
GET	/api/me/sessions	List own sessions

Method	Path	Role
GET	/api/sessions/{id}	Get owned session
POST	/api/coach/start	Start check-in
POST	/api/coach/after-exercise	Per-exercise coach
POST	/api/coach/mid-session	Free-form coach
POST	/api/coach/end	End wrap-up + snapshot
GET	/health	Liveness

10.2 Recommended Client Session Flow

1. POST /api/coach/start (+ optional program_id)
2. User trains
3. POST /api/coach/after-exercise (per exercise; accumulate feedback)
4. POST /api/coach/mid-session (optional)
5. POST /api/coach/end → snapshot
6. POST /api/sessions ← map snapshot + start_time/end_time
7. Later: POST /api/programs/suggest-next

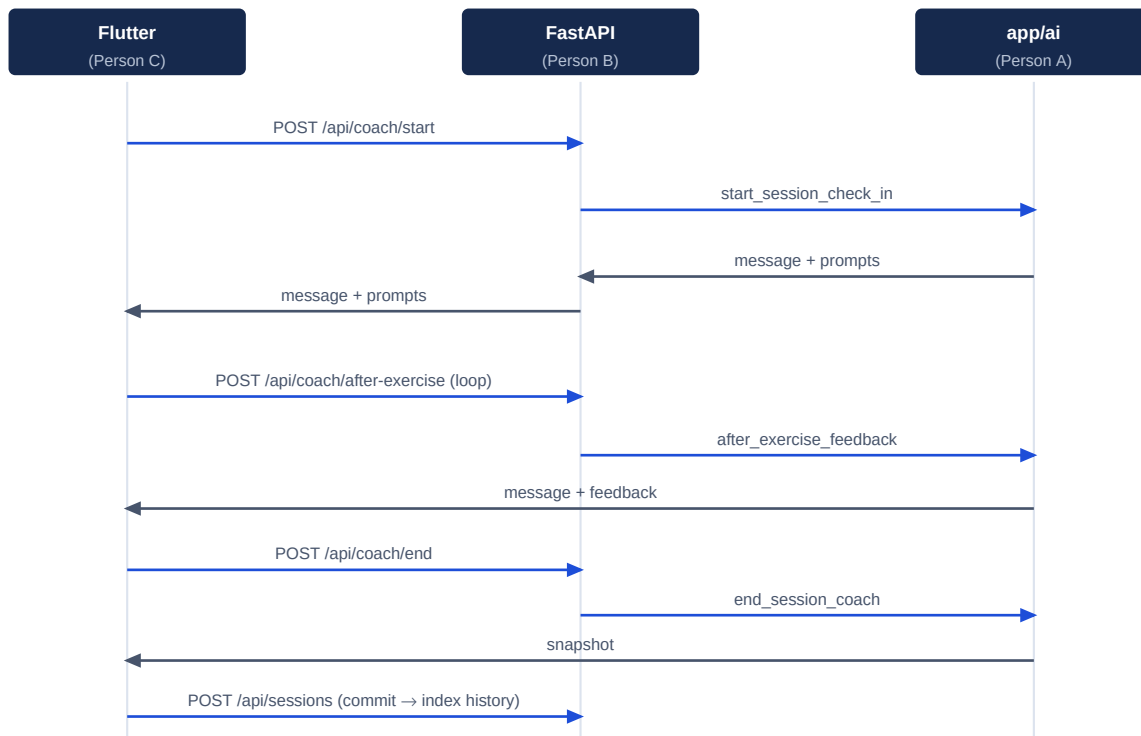


Figure 10.1 — Sequence diagram: a full coaching session end to end

10.3 Common Status Codes

TABLE 10.2 — COMMON STATUS CODES

Code	Typical Meaning
200 / 201	Success (coach/read vs. create)
400	Incomplete profile or a business rule violation (e.g. exercise not on program)
401	Missing or invalid JWT
403	Resource exists but is not owned by the caller
404	Profile / program / session not found
422	Pydantic request validation failure
502	AI failure or an invalid AI-generated payload
500	Unexpected persistence failure (rolled back where applicable)

CHAPTER 11

1 AI Integration Wiring

Routers import AI modules lazily inside their handler functions rather than at module scope. This keeps unit tests lightweight when AI dependencies are stubbed, since importing a router for testing never forces the AI stack to initialize.

TABLE 11.1 — ROUTER-TO-AI WIRING

Router	Wiring
programs.py	<code>_profile_to_context</code> → <code>generate_program</code> / <code>suggest_next_program</code> → <code>validate ProgramCreate</code> → <code>commit</code>
sessions.py	<code>Create ORM session</code> → <code>commit</code> → <code>index_session_history</code> (best-effort)
coach.py	<code>_profile_to_context</code> → live coach functions → return AI result models as JSON

■ IMPLEMENTATION DETAIL

The Person A contract for these calls is defined in **backend/app/ai/CONTRACT.md**, which is the authoritative function signature reference for anyone wiring a new route to the AI layer.

CHAPTER 12

1 Migrations (Alembic)

Migrations run automatically via `entrypoint.sh` (alembic upgrade head) on every API container start. They can also be applied manually:

```
docker compose exec api alembic upgrade head
```

12.1 Representative Migration Chain

TABLE 12.1 — MIGRATION CHAIN

Revision (prefix)	Purpose
f35a1e7d3682 / 02f21795d512	Initial users and profiles
fc245e195b5c	Programs and sessions
a3b8c9d1e2f4	program_id on sessions
b4c5d6e7f8a9	knowledge_chunks (AI)
c5d6e7f8a9b0	exercises + knowledge source
d6e7f8a9b0c1	user_history_chunks
e7f8a9b0c1d2	available_time_minutes on profiles

■ NOTE

AI-related revisions are proposed with Person A's schemas, but Person B owns reviewing and applying them in the running stack — migration authorship and migration ownership are deliberately separated.

CHAPTER 13

1 3 Testing

Unit tests live under `backend/tests/` using `pytest`, `pytest-asyncio`, and an HTTPX ASGI client. AI and database calls are mocked, and heavy optional dependencies (e.g. `fastembed`) are stubbed in `confest.py` so the suite runs without a live AI stack.

TABLE 13.1 — TEST MODULE COVERAGE

Module	Focus
test_profile.py	PATCH profile, validation, auth
test_programs_generate.py	Generate success, incomplete profile, 502, ownership
test_programs_suggest_next.py	Suggest-next wiring
test_sessions.py	Create, ownership, indexing failure soft-fail
test_coach.py	All four coach routes, program_id 403/404, 502
test_review_fixes.py	Review / regression fixes

```
# From backend/ (with deps installed), or inside the api container:
pytest tests/ -v
```

CHAPTER 14

14 Configuration and Verification

14.1 Environment

See `.env.example` and the project `README.md`. The API requires `SECRET_KEY` and `DATABASE_URL`; AI-backed routes additionally require `GROQ_API_KEY`.

14.2 Local Stack

```
cp .env.example .env
# set secrets
docker compose up -d --build
```

TABLE 14.1 — LOCAL RESOURCES

Resource	Location
API	<code>http://localhost:8000</code>
OpenAPI	<code>http://localhost:8000/docs</code>
Health	<code>GET /health → {"status":"ok"}</code>

Optional knowledge ingest (Person A operations, needed for higher-quality AI answers):

```
docker compose exec api python -m app.ai.ingest_docs
docker compose exec api python -m app.ai.ingest_exercises
docker compose exec api python -m app.ai.ingest_pdfs
```

CHAPTER 15

15 Current Status and Next Steps

15.1 Completed (Person B)

- ✓ JWT auth, `/me` profile routes
- ✓ Programs & sessions CRUD with ownership
- ✓ POST `/api/programs/generate` and `suggest-next` (persist)
- ✓ POST `/api/sessions` + best-effort history indexing

- ✓ Live coach HTTP: start / after-exercise / mid-session / end
- ✓ available_time_minutes migration + profile I/O
- ✓ Alembic ownership for AI-related tables
- ✓ Mobile API contracts + pytest suite for B routes

15.2 Remaining (Product)

TABLE 15.1 — REMAINING WORK BY OWNER

Owner	Next Work
Person A (optional)	Full Docker smoke suite when environment allows
Person B	Optional polish: pass available_time_minutes into AI ProfileContext if the product requires it
Person C	Flutter screens for profile, programs, live session prompts; optional voice around text APIs

CHAPTER 16

16 Related Documents

TABLE 16.1 — RELATED DOCUMENTS

Document	Audience
docs/api-contracts.md	Person C — endpoint contracts
docs/AI_COACHING_LAYER.md	Person A — AI/RAG technical documentation
backend/app/ai/CONTRACT.md	Person B — AI function call contract
.env.example	Environment template
README.md	Project entry point

CHAPTER 17

17 Glossary

TABLE 17.1 — GLOSSARY

Term	Meaning
JWT	JSON Web Token used as the Bearer access credential
Owner-scoped	Resource access limited to current_user.id
ProgramCreate	Validated plan payload persisted as a Program

Term	Meaning
SessionSnapshot	Structured finished-session payload produced by end-coach / history indexing
Live coach	In-session HTTP turns returning messages and UI prompts
Best-effort indexing	History embedding that runs after session commit; failure does not undo the session
OpenAPI	Interactive schema at /docs, generated automatically from the FastAPI routers

Conclusion. The backend API layer delivers a complete, owner-scoped, and AI-orchestrated HTTP surface on top of the existing domain foundation. Authentication, CRUD, AI generation, live coaching, and history indexing are all implemented, migrated, documented, and covered by an automated test suite. The remaining work is product polish rather than architectural gaps, and the API is ready to serve as the stable contract for the Flutter client.

End of document — AI Fitness Coach, Backend API Layer Technical Specification, Version 1.0, July 2026.